Smart Commerce - Supervisor Q&A

Inventory intelligence, low-stock alert, AI chatbot, RAG, Gemini API, and demand forecasting

Use these answers for supervisor questions, viva discussion, and project presentation preparation.

1. Question: What do you mean by Inventory Intelligence? How does it work?

Inventory Intelligence means using product stock data, sales information, and simple analysis rules to manage inventory more smartly. A normal inventory system only stores product quantity, but my system helps the admin or vendor understand which products are available, which products are low in stock, which products are out of stock, and which products may need restocking.

It works by storing stock and reorder point values for each product. The system compares the current stock with the reorder point. If the stock is zero, it shows out of stock. If the stock is less than or equal to the reorder point, it shows low stock. Otherwise, it shows in stock. This helps the business make better stock decisions instead of checking everything manually.

2. Question: How did you implement Inventory Intelligence?

I implemented Inventory Intelligence by combining product stock data, reorder rules, inventory logs, dashboard alerts, and demand forecasting.

In the product model, each product has stock and reorderPoint fields. The backend provides an inventory overview API that returns product name, SKU, current stock, reorder point, category, and status. The frontend displays this data in the admin and vendor inventory pages.

The system also supports stock adjustment. When an admin or vendor increases or decreases stock, the system stores the previous stock, new stock, changed quantity, reason, and user information in inventory logs. Because of this, stock movement can be tracked later.

3. Question: How is the low-stock alert given?

The low-stock alert is generated automatically by comparing the product's current stock with the alert threshold or reorder point.

The logic is simple: if stock is zero, the system marks it as out of stock or critical. If stock is less than or equal to the reorder point, the system marks it as low stock. If stock is higher than the reorder point, it is shown as in stock.

For example, if a product has stock 4 and the alert level is 7, then 4 is below 7, so the system shows a low-stock alert. This alert is shown in the admin and vendor dashboard as warning badges and low-stock product lists.

4. Question: Which technologies are used to identify low stock and show notification?

For low-stock identification, I used Node.js, Express.js, TypeScript, MongoDB, and Mongoose. MongoDB stores product stock and reorder point values. The Express backend checks those values using rule-based conditions.

For showing the alert or notification in the interface, I used Next.js, React, TypeScript, Tailwind CSS, badge components, dashboard alert sections, and toast messages. The notification is shown inside the web dashboard, not as SMS or email.

So the backend identifies the low-stock products, and the frontend displays the alert to the admin or vendor.

5. Question: How did you implement the AI-based customer support chatbot?

I implemented the chatbot as an AI-based customer support feature for the e-commerce platform. The main purpose is to answer customer questions instantly without waiting for manual support.

The chatbot takes the user's natural language question, processes the text, retrieves relevant project or product information, and then generates a helpful response. It can answer questions about product availability, price, checkout, order tracking, PDF receipt, dashboard features, inventory, low-stock alert, demand forecasting, demo accounts, and setup information.

This improves the customer experience because users can get quick help directly inside the platform.

6. Question: How does the AI chatbot work?

The chatbot works in a few steps. First, the user types a question in the chatbot widget. Second, the question is sent to the chatbot API. Third, the system processes the text and identifies the user's intent. Fourth, it retrieves relevant context from project knowledge or product data. Finally, the AI model generates a natural answer based on that context.

Because users can ask questions in normal language, the chatbot is more flexible than a fixed FAQ page. It supports customer service by answering repeated questions quickly.

7. Question: Can you say you used RAG and Gemini API in the chatbot?

Yes, if the implementation includes Gemini API and retrieval-based context, I can explain it as a RAG-based AI chatbot.

RAG means Retrieval-Augmented Generation. In this approach, the system first retrieves relevant information from the project knowledge base, product data, or stored support content. Then the retrieved context and the user's question are sent to the Gemini API. Gemini generates the final human-like answer using that context.

Because of RAG, the chatbot does not depend only on general AI knowledge. It answers based on my own project data, which makes the response more accurate and project-specific.

8. Question: Why did you use RAG for the chatbot?

I used RAG because a normal AI model can sometimes give general or incorrect answers. RAG helps the chatbot first collect relevant information from my project data and then generate the answer using that context.

This makes the chatbot more reliable for customer support. For example, if the user asks about stock, checkout, low-stock alert, or demand forecast, the system can use project-specific context before generating the answer.

9. Question: Is ChromaDB necessary to build a RAG chatbot?

No, ChromaDB is not mandatory for RAG. ChromaDB is only one option for storing and searching embeddings.

For RAG, the main requirements are a data source, a retrieval system, an AI model, and a prompt that sends the retrieved context to the model. The data source can be MongoDB, JSON files, text files, PDFs, or a vector database.

Since my project already uses MongoDB, I can retrieve relevant product and project data from MongoDB and pass that context to Gemini API. For a larger system, I can improve retrieval with ChromaDB, FAISS, Pinecone, Weaviate, PostgreSQL with pgvector, or MongoDB Atlas Vector Search.

10. Question: What technologies did you use to implement demand forecasting?

For demand forecasting, I used Node.js, Express.js, TypeScript, MongoDB, Mongoose, a Python forecasting script, Next.js, React, and Recharts.

The backend reads active products and previous non-cancelled orders from MongoDB. Then it groups product sales into recent weekly history. This prepared sales history is sent to a Python script. The Python script calculates predicted demand using weighted average, recent sales trend, trend labels such as rising, stable, or cooling, and a confidence score.

The result is returned to the backend as JSON and displayed in the admin demand forecast page using charts and a ranking table. This helps the admin understand which products may need more stock in the future.

11. Question: Is the demand forecasting deep learning?

No, my current demand forecasting is not deep learning. It is an explainable forecasting method based on previous sales history, weighted average, trend analysis, and confidence scoring.

I used this approach because it is easy to explain, suitable for my project level, and useful for business decision support. In the future, it can be improved with advanced machine learning or deep learning models.